

SYSTEM SPECIFICATION: COGNITIVE REPOSITORY ARCHITECTURE

Document Target: Autonomous Planning & Execution Agents **Role:** Injected Structural & Operational Protocol **Repository Model:** 5+1 Tier Unified Cognitive-Engineering Memory Architecture

1. Core Architectural Invariants

As the planning agent, you must enforce the separation of cognitive memory layers across the filesystem. Adhere strictly to the following invariants:

- 1. **Sensory Immutability (Tier 1):** Never modify, overwrite, or delete assets in 01_raw_sources/. All raw inputs are read-only sources of truth.
- 2. **Human-Machine Decoupling (Tier 2 vs. Tier 3):** Human-readable knowledge lives in 02_master_wiki/ (.md). Machine-readable retrieval chunks live in 03_recall_cache/ (.jsonl, vectors). Never dump raw tabular JSONL or vector embeddings into the wiki layer.
- 3. **Procedural vs. Episodic Separation (Tier 4 vs. Tier 5):** Executable skills, scripts, and policies belong in 04_skills_runtime/. Trajectory traces, error logs, and verifier evaluations belong in 05_episodic_logs/. Never place runtime logic inside logging directories or vice versa.
- 4. **Metacognitive Priming:** Before initiating multi-step tasks, read MAP.md or parse manifest.jsonl to resolve entity references rather than performing blind recursive scans across the filesystem.

2. Directory Schema & Access Control Matrix

📁 vault_root/	
├── 📄 MAP.md	# Master human index & linked
concept graph (Read-Heavy / Agent Update)	
├── 📊 manifest.jsonl	# Machine registry with
sha256 hashes & paths (Append / Sync)	
├── 📁 01_raw_sources/	# TIER 1: COLD SENSORY
ARCHIVE (Access: READ-ONLY)	
└── 📄 pdf/	# Source whitepapers, specs,
data sheets (*.pdf)	
└── 🎬 media/	# Raw recordings, screen
captures, audio (*.mp4, *.wav, *.png)	
└── 📝 text/	# Raw scraped web text,
transcripts, dumps (*.txt, *.html)	
├── 📁 02_master_wiki/	# TIER 2: SEMANTIC MEMORY
VAULT (Access: READ / WRITE)	
└── 💡 concepts/	# Atomic, linked knowledge

```

notes (*.md)
|   | — 🏠 architectures/           # System blueprints and
interface contracts (*.md)
|   | — 🏷️ entities/                 # Schema registries, API
models, entity tables (*.md)
|
| — 📁 03_recall_cache/             # TIER 3: WORKING MEMORY
CACHE (Access: REBUILD / OVERWRITE)
|   | — ⚡ jsonl/                   # Chunked, pre-tokenized
passages (*.jsonl)
|   | — 📐 vectors/                 # Dense vector index
checkpoints (*.bin, *.faiss, *.hnsw)
|   | — 🔑 kv_store/                # Key-value fast entity
lookup tables (*.db, *.json)
|
| — 📁 04_skills_runtime/           # TIER 4: PROCEDURAL
REPERTOIRE (Access: READ / VERSION-CONTROL)
|   | — 🎯 prompt_skills/           # Modular skill files with
frontmatter (*.md, *.yaml)
|   | — ⚙️ runtimes/                 # Standalone Python/Shell
automation routines (*.py, *.sh)
|   | — 🛡️ policies/                 # Validation guards, retry
policies, schemas (*.json, *.yaml)
|
| — 📁 05_episodic_logs/           # TIER 5: EPISODIC TRAJECTORY
STORE (Access: APPEND-ONLY)
|   | — 🕒 trajectories/             # Timestamped run logs:
inputs, calls, outputs (*.jsonl)
|   | — ⚖️ rlvr_verifiers/           # Evaluator rewards,
assertions, benchmark scores (*.jsonl)
|   | — 🧹 hygiene_reports/         # Linter results, broken link
audits, hash mismatches (*.md)

```

3. Tier Operational Specifications

Root: Metacognitive Index (MAP.md & manifest.jsonl)

- **MAP.md:** Must maintain top-level navigation links using Markdown cross-references to key notes in 02_master_wiki/ and entry points in 04_skills_runtime/.
- **manifest.jsonl:** Each record must adhere to the JSON schema:

```

{"id": "SRC-0042", "path": "01_raw_sources/pdf/spec_v1.pdf",
"tier": 1, "sha256": "...", "last_modified":
"2026-09-03T17:00:00Z", "linked_wiki":
"02_master_wiki/architectures/spec_v1.md"}

```

Tier 1: 01_raw_sources/ (Cold Archive)

- **Write Policy:** Write-once upon raw intake. Never mutate or reformat existing files.
- **Naming Convention:** YYYYMMDD_source_title.[ext] (lowercase, snake_case).

Tier 2: 02_master_wiki/ (Semantic Memory)

- **Write Policy:** Incremental updates. Every document must use frontmatter:

```
---
id: wiki_concept_name
tier: 2
created: YYYY-MM-DD
updated: YYYY-MM-DD
sources: [ "01_raw_sources/pdf/source_doc.pdf" ]
tags: [ memory, architecture, rlvr ]
---
# Title
## Context & Definition
...
## References
- [[related_concept_note]]
```

- **Format:** Pure Markdown. Disallow embedded binaries or serialized data blobs.

Tier 3: 03_recall_cache/ (Working Memory Accelerator)

- **Write Policy:** Agent-generated during build or indexing steps. May be deleted and regenerated safely.
- **Chunk Schema (jsonl/):**

```
{ "chunk_id": "CHK-8901", "parent_doc":
"02_master_wiki/concepts/rlvr.md", "tokens": 256, "content":
"...", "metadata": { "tier": 2, "topic": "rlvr" } }
```

Tier 4: 04_skills_runtime/ (Procedural Memory)

- **Write Policy:** Version-controlled tooling and instructions.
- **Skill Format (prompt_skills/):** Standard modular prompt format containing purpose, required inputs, tool dependencies, and output schemas.
- **Runtime Code (runtimes/):** Self-contained, non-interactive scripts with standard exit codes (0 = success, non-zero = error).

Tier 5: 05_episodic_logs/ (Episodic Memory)

- **Write Policy:** Append-only during agent task execution.
- **Trajectory Schema (trajectories/YYYYMMDD_session.jsonl):**

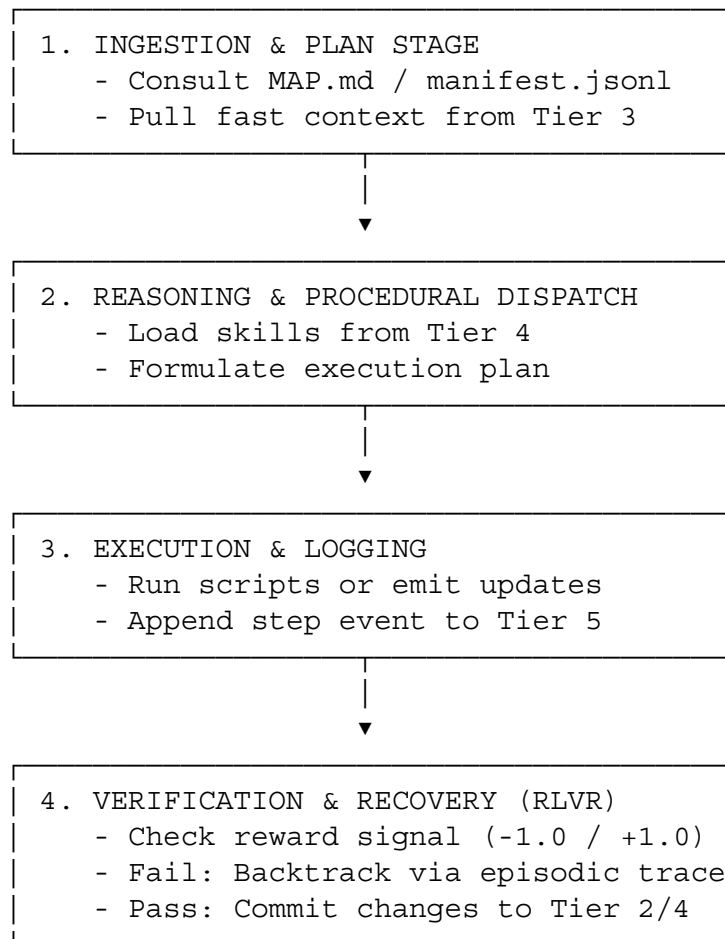
```
{ "timestamp": "2026-09-03T17:01:35Z", "step": 1, "task_id":
```

```
"TASK-104", "prompt_hash": "...", "tool_call": "run_linter",  
"exit_code": 0, "response_snippet": "OK"}
```

- **RLVR Log Schema (rlvr_verifiers/):**

```
{"timestamp": "2026-09-03T17:01:40Z", "task_id": "TASK-104",  
"verifier_id": "syntax_test", "reward": 1.0, "feedback": "All  
assertions passed."}
```

4. Agent Execution & Planning Protocols



Operational Steps for the Planning Agent

1. **Step 1: Locating Information (Read Pass)**

- When asked a domain question, check 03_recall_cache/ first for relevant chunks.
- If full context is required, resolve the source path via MAP.md and read the relevant note in 02_master_wiki/.
- Only query 01_raw_sources/ if source verification or citation check is explicitly

requested.

2. Step 2: Scaffolding or Modifying Repo Content (Write Pass)

- When processing new source material:
 1. Place the original asset into 01_raw_sources/<type>/.
 2. Create or update the corresponding synthesized note in 02_master_wiki/<category>/.
 3. Regenerate the corresponding search chunks in 03_recall_cache/jsonl/.
 4. Append the record to manifest.jsonl and update cross-links in MAP.md.

3. Step 3: Executing Tools & Self-Correction (RLVR Loop)

- Retrieve relevant executable scripts or skills from 04_skills_runtime/.
- Stream execution status and tool payloads into 05_episodic_logs/trajectories/.
- If a step fails (reward: -1.0 or non-zero exit code):
 - Do **not** discard the error context.
 - Ingest the error trace from Tier 5 into active Working Memory.
 - Search 04_skills_runtime/policies/ for documented recovery heuristics.
 - Attempt remediation and log the retry attempt.

5. Repository Scaffolding Script

When initializing or verifying an empty or unaligned repository, execute this shell sequence to enforce structural compliance:

```
#!/usr/bin/env bash
set -euo pipefail
```

```
# 1. Create directory topology
mkdir -p 01_raw_sources/{pdf,media,text}
mkdir -p 02_master_wiki/{concepts,architectures,entities}
mkdir -p 03_recall_cache/{jsonl,vectors,kv_store}
mkdir -p 04_skills_runtime/{prompt_skills,runtimes,policies}
mkdir -p
05_episodic_logs/{trajectories,rlvr_verifiers,hygiene_reports}
```

```
# 2. Touch metacognitive root anchors if missing
if [ ! -f "MAP.md" ]; then
  cat << 'EOF' > MAP.md
# Master Repository Ontology Map
```

```
## 01. Raw Sources
- Link to raw files cataloged in `manifest.jsonl`.
```

```
## 02. Conceptual Wiki
- Concepts: `02_master_wiki/concepts/`
- Architectures: `02_master_wiki/architectures/`
- Entities: `02_master_wiki/entities/`
```

```
## 03. Procedural Runtimes & Skills
- Prompt Skills: `04_skills_runtime/prompt_skills/`
```

```
- Execution Scripts: `04_skills_runtime/runtimes/`  
EOF  
fi  
  
if [ ! -f "manifest.jsonl" ]; then  
    touch manifest.jsonl  
fi  
  
echo "Repository structure verified and initialized."
```